



République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Sfax

Ecole Nationale d'Electronique et des
Télécommunications de Sfax



Ingénierie Génie
Informatique Industrielle

2GII3/



Mini Projet : Test et validation

Réaliser par :

: Nawres barhoumi /Amal bouabid

2GII3

2023/2024

Test et validation :

Le test et la validation sont des processus systématiques visant à évaluer la conformité, la qualité et la performance d'un logiciel ou d'un système par rapport à des critères prédéfinis, qu'ils soient spécifiés par les exigences des utilisateurs, les besoins fonctionnels, ou les normes de l'industrie. Ces processus cohérents à exécuter le logiciel, à analyser ses résultats et à identifier tout écart par rapport aux attentes, dans le but d'assurer la fiabilité, la fonctionnalité et l'adéquation globale du produit final.

Question 1 :

Pour le développement de cette application, nous avons opté pour l'utilisation des langages de programmation HTML, CSS et JavaScript. Ce choix a été fait en tenant compte des exigences fonctionnelles et techniques du projet, ainsi que des avantages spécifiques offerts par chaque langage :

- **HTML (HyperText Markup Language)** : HTML est un langage de balisage utilisé pour la création et la structuration du contenu web.
- **CSS (Cascading Style Sheets)** : CSS est un langage de feuilles de style utilisé pour définir la présentation visuelle des pages web.
- **JavaScript** : JavaScript est un langage de programmation de scripts côté client qui permet d'ajouter des fonctionnalités interactives et dynamiques aux pages web.

Code :

```
<!DOCTYPE html>
<html lang="en">
<head>

  <meta charset="UTF-8">

  <title>test et validation</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f0f0f0;
      margin: 0;
      padding: 0;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
  </style>
</head>
<body>
  <div>
    <h1>test et validation</h1>
  </div>
</body>
</html>
```

```

        .container {
            background-color: #fff;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
        }

        input[type="number"] {
            width: 100px;
            padding: 8px;
            margin-right: 10px;
        }

        #result {
            margin-top: 20px;
            padding: 10px;
            border: 1px solid #ccc;
            border-radius: 4px;
        }

        .green {
            background-color: lightgreen;
        }

        .red {
            background-color: lightcoral;
        }
    </style>

</head>
<body>

    <div class="container">
        <label for="moyenne">Moyenne:</label>
        <input type="number" id="moyenne" min="-100" max="20" step="0.01"
oninput="calculateResult()">
        <div id="result"></div>
    </div>

    <script>
        function calculateResult() {
            var moyenne =
parseFloat(document.getElementById("moyenne").value);
            var resultElement = document.getElementById("result");

```

```

var resultat;
var backgroundColor;

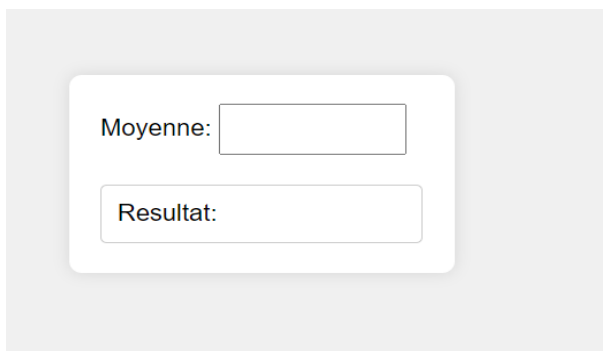
if (!isNaN(moyenne) && moyenne >= 0 && moyenne < 10) {
    resultat = "Rattrapage";
    backgroundColor = "green";
} else if (!isNaN(moyenne) && moyenne >= 10 && moyenne <= 20) {
    resultat = "ADMIS";
    backgroundColor = "green";
} else {
    resultat = "Insérez une moyenne entre 0 et 20";
    backgroundColor = "red";
}

resultElement.textContent = resultat;
resultElement.className = backgroundColor === "green" ? "green" :
"red";
}
</script>

</body>
</html>

```

Résultat →



The image shows a web form with a light gray background. Inside, there is a white rounded rectangle containing two input fields. The first field is labeled 'Moyenne:' and is empty. The second field is labeled 'Resultat:' and is also empty.

Les conditions :

-E1 :Si $(0 \leq \text{Moy} < 10)$ → Rattrapage :

Moyenne: 5

Rattrapage

E2 : Si $(10 \leq \text{Moy} \leq 20)$ $\rightarrow$ ADMIS :

Moyenne: 11.5

ADMIS

-E3 : Si $(\text{Moy} < 0)$ ou $(\text{Moy} > 20)$ $\square$ insérez une moyenne entre 0 et 20 :

Left screenshot: Moyenne: 22, Insérez une moyenne entre 0 et 20

Right screenshot: Moyenne: -2.35, Insérez une moyenne entre 0 et 20

Question2 :

Pour tester le logiciel que nous avons développé, nous avons utilisé les tests unitaires (les tests automatique) pour vérifier le bon fonctionnement des unités individuelles de code :

Les tests logiciels sont un moyen :

-d'évaluer la qualité du logiciel.

-réduire le risque de défaillance de ce logiciel en cours de fonctionnement.

Ces outils offrent généralement des fonctionnalités telles que l'organisation des tests, l'exécution par lots, la gestion des assertions, la mise en place et le nettoyage des environnement de test ,le reporting des résultats , etc. ..

-les étapes que nous avons fait pour rendre ce travail possible :

-téléchargement du logiciel python :

Python est un langage de programmation informatique généraliste. Contrairement à HTML, CSS ou JavaScript, son usage n'est donc pas limité au développement web. Il peut être utilisé pour tout type de programmation et de développement logiciel.



-téléchargement du sélénium :

Sélénium Web Driver est un Framework web qui vous permet d'exécuter des tests multi-navigateurs. Cet outil est utilisé pour automatiser les tests d'applications Web pour vérifier qu'ils fonctionnent correctement.



-Configuration de l'environnement de développement :

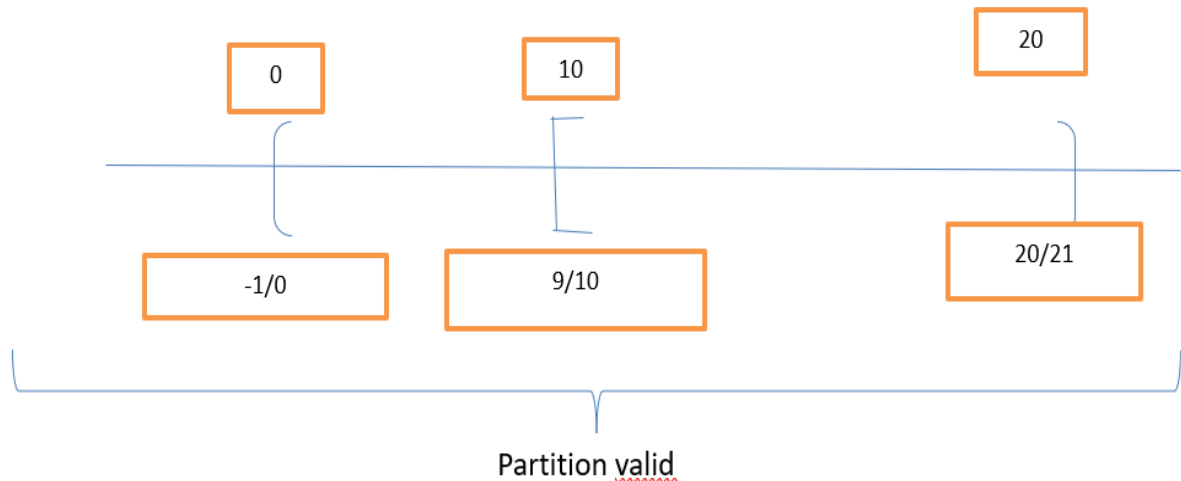
-On va ouvrir Visual Studio Code, Puis on fait l'installation de l'extension Python pour Visual Studio Code pour bénéficier de fonctionnalités spécifiques à Python et on va Installer Selenium :

-En Utilisant pip, le gestionnaire de packages de Python, pour installer Selenium. On va ouvrir le terminal et on va taper cette commande :

```
pip install selenium
```

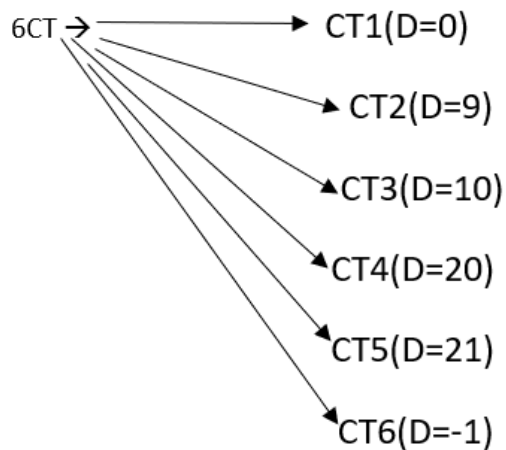
-On va écrire le test Sélénium en utilisant le langage de programmation Python (en utilisant la bibliothèque sélénium) :

On va utiliser AVL (Analyse valeurs limites à 2 valeurs) :



Partition valide : sont les intervalles à un résultat attendu. (Ici tous les intervalles ont un résultat attendu).

Donc nous avons 6 cas de test pour avoir la clôture (pas=1) :



Couverture (Cv)=100%

Cas de test 1 : (D=0)

```

import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

class TestResultCalculation(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.implicitly_wait(30)
        self.driver.get("file:///C:/Users/user/Desktop/projet.html")

    def tearDown(self):
        self.driver.quit()

    def test_calculate_result_ratrapage(self):

        average_input = self.driver.find_element(By.ID, "moyenne")
        average_input.send_keys(0)

        time.sleep(1)

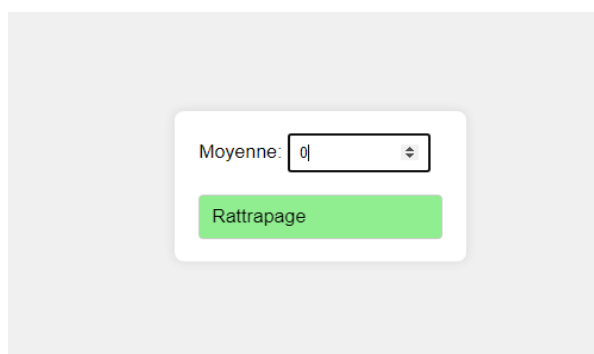
        result_element = self.driver.find_element(By.ID, "result")
        result_text = result_element.text

        self.assertEqual(result_text, "Ratrapage")

if __name__ == "__main__":
    unittest.main()

```

-Résultat :




```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

DevTools listening on ws://127.0.0.1:58407/devtools/browser/8b07ff3f-9b2f-4c16-8c02-12d363cb57b6
.
-----
Ran 1 test in 9.746s

OK
```

-Cas de test 2 : (D=9)

```
class TestResultCalculation(unittest.TestCase):
    def setUp(self):
        self.driver.get("file:///C:/Users/user/Desktop/projet.html")

    def tearDown(self):
        self.driver.quit()

    def test_calculate_result_rattrapage(self):

        average_input = self.driver.find_element(By.ID, "moyenne")
        average_input.send_keys(9)

        time.sleep(1)

        result_element = self.driver.find_element(By.ID, "result")
        result_text = result_element.text

        self.assertEqual(result_text, "Rattrapage")

if __name__ == "__main__":
    unittest.main()
```



```

DevTools listening on ws://127.0.0.1:58462/devtools/browser/64ebd404-0932-4997-9474-2029e58a6f4e
.
-----
Ran 1 test in 19.838s

OK

user@DESKTOP-D170V4V MINGW64 ~
$ 

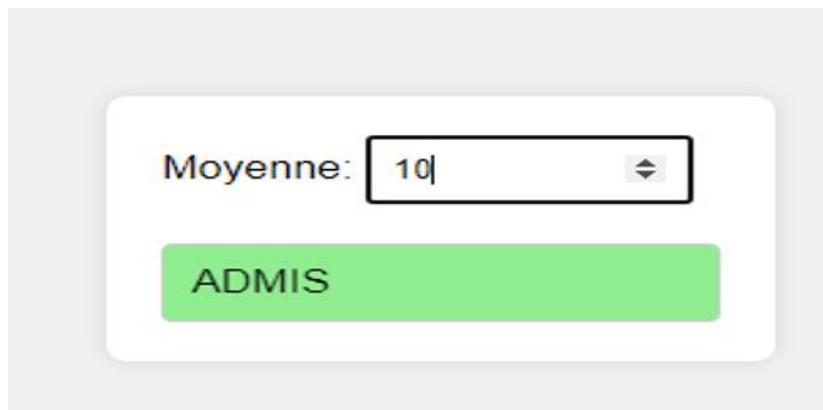
```

-Cas de test 3:(D=10)

```

C: > Users > user > Desktop > set.py > ...
1  import unittest
2  from selenium import webdriver
3  from selenium.webdriver.common.by import By
4  import time
5
6  class TestResultCalculation(unittest.TestCase):
7
8      def setUp(self):
9          self.driver = webdriver.Chrome()
10         self.driver.implicitly_wait(30)
11         self.driver.get("file:///C:/Users/user/Desktop/projet.html")
12
13     def tearDown(self):
14         self.driver.quit()
15
16
17
18     def test_calculate_result_admis(self):
19         # Enter average 17
20         average_input = self.driver.find_element(By.ID, "moyenne")
21         average_input.send_keys(10)
22
23         # Wait for JavaScript to update the result
24         time.sleep(1)
25
26         # Get the result text
27         result_element = self.driver.find_element(By.ID, "result")
28         result_text = result_element.text
29
30         # Assert that the result is "ADMIS"
31         self.assertEqual(result_text, "ADMIS")
32
33
34
35 if __name__ == "__main__":
36     unittest.main()

```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

DevTools listening on ws://127.0.0.1:58734/devtools/browser/67acc70d-8dfb-4aaf-97df-7cf3e1422b3f
.
-----
Ran 1 test in 10.056s

OK

user@DESKTOP-D170V4V MINGW64 ~
$
```

-Cas de test 4 :(D=20)

```

ers > user > Desktop > set.py > testResultCalculation > test_calculate_result_admis

import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

class TestResultCalculation(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.implicitly_wait(30)
        self.driver.get("file:///C:/Users/user/Desktop/projet.html")

    def tearDown(self):
        self.driver.quit()

    def test_calculate_result_admis(self):
        # Enter average 17
        average_input = self.driver.find_element(By.ID, "moyenne")
        average_input.send_keys([20])

        # Wait for JavaScript to update the result
        time.sleep(10)

        # Get the result text
        result_element = self.driver.find_element(By.ID, "result")
        result_text = result_element.text

        # Assert that the result is "ADMIS"
        self.assertEqual(result_text, "ADMIS")

if __name__ == "__main__":
    unittest.main()

```

The screenshot shows a web interface with a label 'Moyenne:' followed by a text input field containing the value '20'. Below the input field is a green rectangular button with the text 'ADMIS' in white capital letters. The entire form is set against a light gray background.

```

-----
Ran 1 test in 21.276s

OK

```

-Cas de test 6 :(-1)

```

import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By

class TestResultCalculation(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.implicitly_wait(30)
        self.driver.get("file:///C:/Users/user/Desktop/projet.html")

    def tearDown(self):
        self.driver.quit()

    def test_calculate_result_invalid_average(self):
        # Enter invalid average -5
        average_input = self.driver.find_element(By.ID, "moyenne")
        average_input.send_keys[-1]

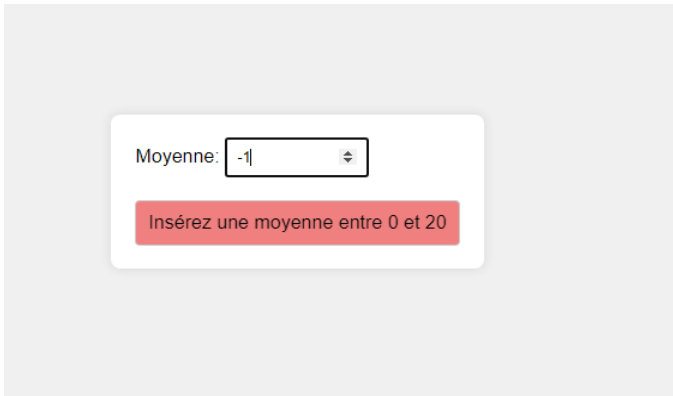
        # Wait for JavaScript to update the result
        time.sleep(1)

        # Get the result text
        result_element = self.driver.find_element(By.ID, "result")
        result_text = result_element.text

        # Assert that the result is the error message
        self.assertEqual(result_text, "Insérez une moyenne entre 0 et 20")

if __name__ == "__main__":
    unittest.main()

```



Moyenne:

Insérez une moyenne entre 0 et 20

```

DevTools listening on ws://127.0.0.1:59016/devtools/browser/6712376d-86c8-4440-b7c9-2651e5592e29
.
-----
Ran 1 test in 19.197s

OK

```

-Cas de test 5:(21)

```

ers / user / Desktop / testpy / testresultcalculation / test_calculate_result_invalid_average
import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

class TestResultCalculation(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.implicitly_wait(30)
        self.driver.get("file:///C:/Users/user/Desktop/projet.html")

    def tearDown(self):
        self.driver.quit()

    def test_calculate_result_invalid_average(self):

        average_input = self.driver.find_element(By.ID, "moyenne")
        average_input.send_keys([21])

        time.sleep(10)

        result_element = self.driver.find_element(By.ID, "result")
        result_text = result_element.text

        self.assertEqual(result_text, "Insérez une moyenne entre 0 et 20")

if __name__ == "__main__":
    unittest.main()

```

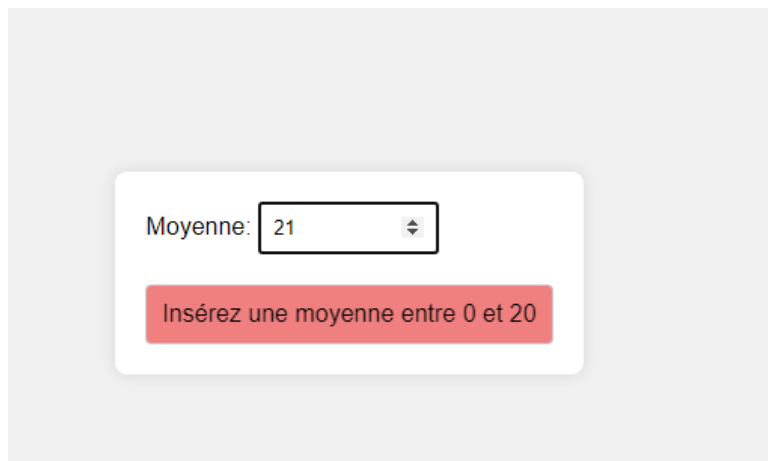
[FMS](#)
[OUTPUT](#)
[DEBUG CONSOLE](#)
[TERMINAL](#)
[PORTS](#)

```

DevTools listening on ws://127.0.0.1:59056/devtools/browser/f7c83c10-02e6-4258-b4bb-aa3222029596
.
-----
Ran 1 test in 19.430s

OK

```



-on va tester le logiciel négativement :

Tests - :

Cas de test 1 : -5

```
users > user > Desktop > set.py > testResultCalculation > test_calculate_result_invalid_average
import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

class TestResultCalculation(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.implicitly_wait(30)
        self.driver.get("file:///C:/Users/user/Desktop/projet.html")

    def tearDown(self):
        self.driver.quit()

    def test_calculate_result_invalid_average(self):

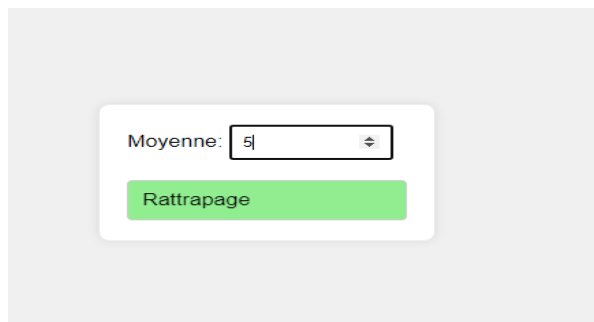
        average_input = self.driver.find_element(By.ID, "moyenne")
        average_input.send_keys(5)

        time.sleep(10)

        result_element = self.driver.find_element(By.ID, "result")
        result_text = result_element.text

        self.assertEqual(result_text, "Insérez une moyenne entre 0 et 20")

if __name__ == "__main__":
    unittest.main()
```



```
-----
Ran 1 test in 15.491s
```

```
FAILED (failures=1)
```

- Cas admis :

```
import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By

class TestResultCalculation(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.implicitly_wait(30)
        self.driver.get("file:///C:/Users/user/Desktop/projet.html")

    def tearDown(self):
        self.driver.quit()

    def test_calculate_result_rattrapage(self):

        average_input = self.driver.find_element(By.ID, "moyenne")
        average_input.send_keys([5])

        time.sleep(1)

        result_element = self.driver.find_element(By.ID, "result")
        result_text = result_element.text

        self.assertEqual(result_text, "Rattrapage")

if __name__ == "__main__":
    unittest.main()
```




```
Ran 1 test in 15.030s
```

```
FAILED (failures=1)
```

-Cas de test 2 :(17)

```
import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

class TestResultCalculation(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.implicitly_wait(30)
        self.driver.get("file:///C:/Users/user/Desktop/projet.html")

    def tearDown(self):
        self.driver.quit()

    def test_calculate_result_invalid_average(self):

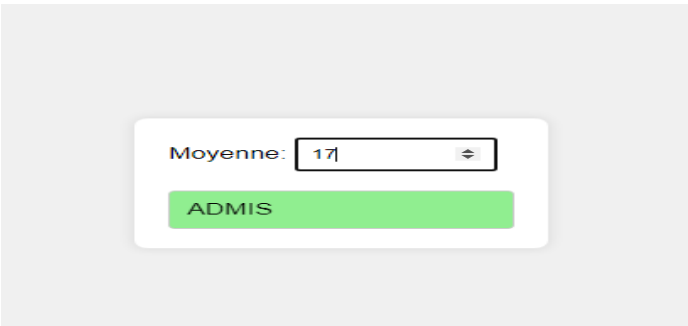
        average_input = self.driver.find_element(By.ID, "moyenne")
        average_input.send_keys(17)

        time.sleep(10)

        result_element = self.driver.find_element(By.ID, "result")
        result_text = result_element.text

        self.assertEqual(result_text, "Insérez une moyenne entre 0 et 20")

if __name__ == "__main__":
    unittest.main()
```



Moyenne:

ADMIS

```
-----
Ran 1 test in 16.803s
```

```
FAILED (failures=1)
```

```
[]
```

- Rattrapage :

```
import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

class TestResultCalculation(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.implicitly_wait(30)
        self.driver.get("file:///C:/Users/user/Desktop/projet.html")

    def tearDown(self):
        self.driver.quit()

    def test_calculate_result_rattrapage(self):

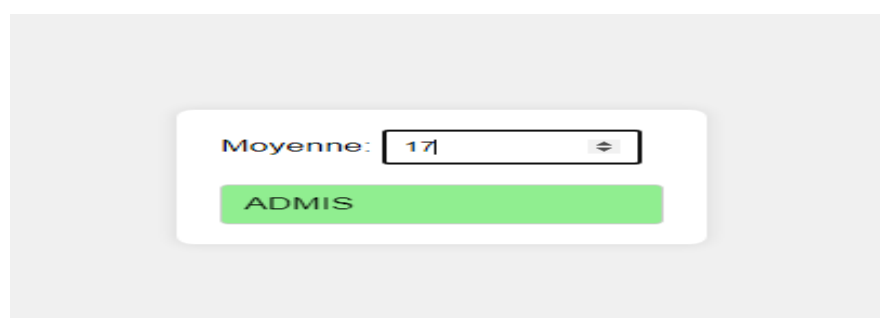
        average_input = self.driver.find_element(By.ID, "moyenne")
        average_input.send_keys([17])

        time.sleep(10)

        result_element = self.driver.find_element(By.ID, "result")
        result_text = result_element.text

        self.assertEqual(result_text, "Rattrapage")

if __name__ == "__main__":
    unittest.main()
```



```
-----
Ran 1 test in 15.094s

FAILED (failures=1)

user@DESKTOP-D170V4V MINGW64 ~
$
```

Cas de test 3 :(-5)

- Admis

```

import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

class TestResultCalculation(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.implicitly_wait(30)
        self.driver.get("file:///C:/Users/user/Desktop/projet.html")

    def tearDown(self):
        self.driver.quit()

    def test_calculate_result_admis(self):

        average_input = self.driver.find_element(By.ID, "moyenne")
        average_input.send_keys(-5)

        |

        time.sleep(10)

        result_element = self.driver.find_element(By.ID, "result")
        result_text = result_element.text

        self.assertEqual(result_text, "ADMIS")

if __name__ == "__main__":
    unittest.main()

```

Moyenne:

Insérez une moyenne entre 0 et 20

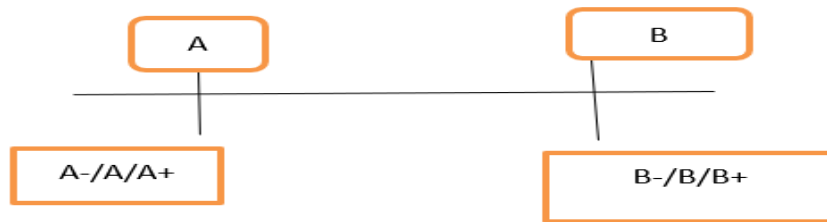
```

-----
Ran 1 test in 16.353s

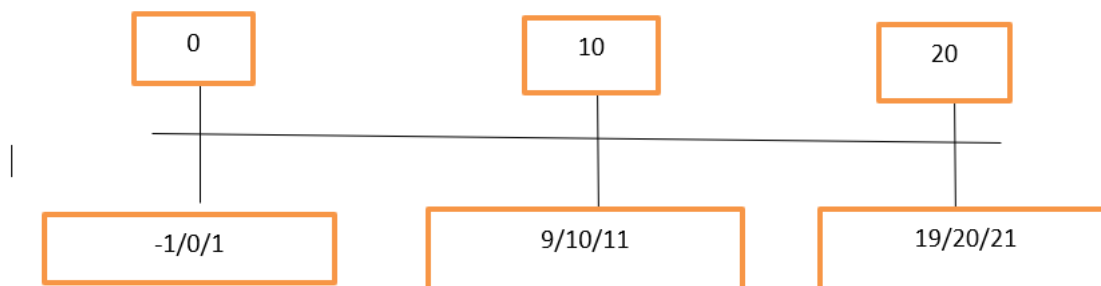
FAILED (failures=1)

```

-Ou bien on va appliquer AVL (a 3 valeurs)(pour garantir qu'on travaille correctement) :(on choisit le pas=1)



-Donc dans le cas de notre application :



-Donc nous avons 9CT pour avoir une couverture 100% sont les suivants :

(D=-1/0 /1/9/10/11/19/20/21).

NB : dans le cadre de notre nous avons choisir juste de travailler sur AVL à 2 valeurs.